

Streaming Communication for Scalable Data Exchange in DYNAMOS

Ruben Stoop
Complex Cyber-Infrastructure (CCI)
University of Amsterdam
Amsterdam, the Netherlands

Abstract—This thesis investigates how streaming communication can be integrated into DYNAMOS, a policy-driven middleware for dynamically composed microservice workflows. The original DYNAMOS data path is built around unary request-response transfers, which creates a bottleneck when large result sets must be moved through generated service chains. The thesis first compares candidate communication mechanisms in a controlled Kubernetes pipeline, using throughput, latency, pacing, correctness, recovery, and resource pressure as evaluation dimensions. It then implements three chunk-aware DYNAMOS data paths: chunked unary transfer, intra-job gRPC streaming, and RabbitMQ Streams at the agent boundary. A local validation on bulk `dataThroughTtp` retrieval shows that all three chunked implementations reduce completion time and peak memory compared with the original classic-unary baseline, with additional checks on `computeToData` and multi-provider `dataThroughTtp`. The results show that streaming is not one design choice but a set of architectural tradeoffs between raw transfer efficiency, temporal decoupling, recovery behaviour, and fit with DYNAMOS’ policy-driven execution model.

Index Terms—streaming communication, microservices, gRPC, event streaming, policy-driven middleware, DYNAMOS

I. INTRODUCTION

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organizations. In such settings, data exchange must adhere to legal agreements, privacy regulations, and organizational policies. These constraints are common in domains such as healthcare, scientific research, and public-sector infrastructures, where unrestricted data exchange is not acceptable.

DYNAMOS (Dynamically Adaptive Microservice-based Operating System) is a data exchange middleware developed by the Complex Cyber-Infrastructure (CCI) group at the University of Amsterdam. The system implements a set of recurring data sharing archetypes that describe how data and computation are distributed among participants under policy constraints. DYNAMOS realizes these archetypes by dynamically composing chains of microservices that execute data sharing requests in compliance with formally evaluated policies [1], [2]. The middleware is designed to be adaptive, allowing architectural and execution decisions to change at runtime in response to policy requirements.

The current implementation of DYNAMOS uses gRPC for inter-service communication and relies primarily on atomic request-response interactions [1]. This communication model

is suitable for small or bounded data transfers but becomes limiting when applied to large-scale or continuous data exchange. In practice, data in distributed systems is often produced and consumed incrementally, for example in sensor data streams, distributed analytics pipelines, or staged processing of large research datasets.

When request-response communication is used in such scenarios, systems must either buffer large volumes of data before transmission or split transfers into multiple independent requests. Buffering increases memory usage and can lead to idle time between producers and consumers. Repeated request-response interactions introduce additional coordination overhead. Both effects negatively impact scalability and resource utilization as data volumes and the number of participating services increase.

Streaming-based communication has been proposed as a solution to these limitations. Streaming allows data to be transferred and processed incrementally, enabling overlap between computation and communication and reducing peak memory usage. gRPC supports several streaming interaction patterns, including client-side, server-side, and bidirectional streaming.

Integrating streaming communication into DYNAMOS is not straightforward. Streaming relies on long-lived connections and continuous data flows, which must be reconciled with DYNAMOS’ policy-driven execution model, dynamic microservice composition, orchestration decisions, and fault handling mechanisms. Without careful design, the introduction of streaming may interfere with policy enforcement or reduce the ability to evaluate and reason about system behavior.

The goal of this thesis is to investigate how streaming communication can be incorporated into DYNAMOS in a way that improves scalability and performance while preserving policy compliance and adaptive behavior. This involves analyzing existing streaming communication mechanisms, designing architectural extensions to the middleware, implementing streaming support, and evaluating the resulting system under realistic workloads.

The main research question addressed in this thesis is:

Main RQ: How can streaming communication be integrated into a dynamically adaptive, policy-driven microservice-based data exchange middleware such as DYNAMOS, while preserving compliance guarantees and improving scalability and performance?

To answer this question, the following sub-research questions are defined:

- **RQ1:** Which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS?
- **RQ2:** How can streaming communication be integrated into a microservice-based architecture in a way that preserves dynamic service composition, policy-driven orchestration, and isolation guarantees?
- **RQ3:** What is the impact of the implemented streaming data paths on scalability, performance, and robustness compared to a traditional request-response approach under realistic DYNAMOS workloads?

II. RELATED WORK

This section reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

A. Streaming communication technologies and interaction models

DYNAMOS is implemented as a dynamically composed microservice system deployed on Kubernetes, using gRPC for inter-service communication and RabbitMQ for asynchronous messaging [1], [2]. This constrains the design space for introducing streaming functionality to a limited set of technologies that are compatible with containerized microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. The original DYNAMOS thesis identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity [1].

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request-response interactions. Starck and Ghofrani show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to

become unevenly distributed across replicas [3]. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualization overhead, with measurable effects on latency and resource utilisation in containerized environments [4].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [5]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes integration [6]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasize that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [7]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. Existing studies are still useful methodologically because they emphasize identical workloads, common measurement points, latency distribution, throughput, resource pressure, and recovery behaviour as recurring benchmark dimensions [8]–[11]. The gap addressed in this thesis is therefore not the invention of a new benchmarking methodology, but the application of those dimensions to dynamically composed, policy-driven middleware with ephemeral execution units.

B. Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [1], [2]. These chains are short-lived by design, which improves isolation

and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. The sidecar pattern is widely used in cloud-native systems to address cross-cutting concerns such as observability, security, and traffic management. Figueira and Coutinho demonstrate how sidecars can be integrated into self-adaptive microservice architectures on Kubernetes, enabling runtime adaptation without violating service modularity [12]. While their work does not explicitly address streaming data transfer, it illustrates how communication behaviour can be adapted dynamically at runtime.

Several middleware frameworks operationalise sidecars as first-class runtime components. Figueira and Coutinho discuss sidecar-based runtimes such as Dapr, which provide a protocol-agnostic communication abstraction supporting service invocation, pub/sub, and state management through a sidecar process [12]. Such runtimes demonstrate that heterogeneous communication paradigms, including streaming and messaging, can be unified behind a stable interface. However, this class of runtimes assumes relatively static service topologies and does not address dynamic chain composition driven by policy evaluation, which limits their applicability as a direct solution for DYNAMOS.

Existing architectural work therefore supports the feasibility of integrating streaming into sidecar-based microservice systems but does not address the combination of long-lived streaming connections with ephemeral, dynamically composed execution chains. This limitation is particularly relevant in systems that combine adaptivity with strict policy enforcement requirements.

C. Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [1], [2]. In the current architecture, policy enforcement is implicitly scoped to bounded request-response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

Neumaier et al. propose a policy-aware architecture for decentralized dataset exchange that emphasizes continuous monitoring and runtime validation against machine-readable policies [13]. While their work does not explicitly target streaming RPC, it introduces an important distinction between one-time policy clearance and ongoing compliance monitoring, which becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiyagarajan et al. present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication,

and RBAC [14]. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how policy enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing streaming into policy-driven middleware such as DYNAMOS.

D. Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request-response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

Reactive microservice architectures emphasize asynchronous communication, non-blocking execution, and fault isolation as mechanisms for improving resilience under variable load [15]. Although this work does not focus on streaming RPC, its principles are directly applicable to streaming workflows, where failures and backpressure must be managed continuously rather than per request.

Studies on high-volume data processing in microservices highlight the importance of explicit flow control and decoupling between producers and consumers. Guntupalli and Vamshi show that event-driven architectures combined with messaging systems can mitigate bottlenecks and stabilize performance under sustained data rates [16]. While these systems typically rely on brokers rather than direct streaming RPC, they reinforce the need for explicit backpressure mechanisms in streaming data paths.

From a systems perspective, Gan and Delimitrou demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments [17]. Their findings underline that resilience and scalability issues often emerge from inter-service communication rather than computation, an effect that is amplified in streaming scenarios.

Existing literature provides well-established principles for resilience and fault tolerance in microservice systems, but these are typically studied either in request-oriented architectures or broker-based streaming platforms. There is limited work examining how these mechanisms interact with direct streaming RPC in dynamically composed microservice chains, leaving questions about how failures and backpressure should be handled in dynamically composed microservice systems.

E. Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction styles in terms of throughput, latency, and resource usage. Studies focused specifically on gRPC streaming show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [4], [18]. Complementary protocol-level comparisons between REST and gRPC further establish that binary, multiplexed transports outperform request-response HTTP under load, which provides additional context for choosing gRPC as the baseline communication layer [19], [20].

Scalability in containerized streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerized microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [10]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [3].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

III. RQ1: TRANSPORT BENCHMARK DESIGN AND RESULTS

This section addresses *RQ1*: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS? It combines a literature-backed comparison of candidate mechanisms with a controlled benchmark design that establishes a gRPC reference baseline before broker-based alternatives are added.

A. gRPC streaming

According to the official gRPC documentation, gRPC is based on service definitions in Protocol Buffers and supports four RPC styles: unary, server-streaming, client-streaming, and bidirectional streaming. For streaming calls, message ordering is preserved within each individual RPC, while deadlines, cancellation, metadata, and both synchronous and asynchronous APIs are supported by the runtime [21]. These properties make gRPC streaming the most direct extension of the current DYNAMOS communication model.

For microservice chains with predominantly point-to-point data transfer, client-streaming is the most natural first candidate because it lets a sender incrementally push chunks or records to a single downstream consumer. Bidirectional streaming becomes relevant when the receiver should provide progress feedback, pacing information, or explicit acknowledgements during transfer. The main advantages of gRPC streaming are low protocol overhead, direct reuse of protobuf contracts, and the absence of an additional broker layer. The disadvantages are that sender and receiver remain temporally coupled, durable replay is not provided by default, and backlog handling or crash recovery must be implemented at application level. Even so, prior work on asynchronous gRPC streaming demonstrates that carefully designed protocols can support efficient and even exactly-once style delivery patterns, which makes gRPC a strong experimental baseline rather than merely the status-quo option [4], [18], [19].

B. Apache Kafka

Apache Kafka is officially described as an open-source distributed event-streaming platform for high-performance pipelines, streaming analytics, data integration, and mission-critical applications. Its published core capabilities emphasise high throughput, durable storage, high availability, built-in stream processing, and large-scale horizontal scalability [22]. In architectural terms, Kafka exposes an append-only log abstraction in which messages are written to topics and partitions, allowing replay and independent consumption by multiple downstream readers [23].

Kafka is therefore the strongest candidate when temporal decoupling, replayability, or large consumer backlogs matter more than direct point-to-point simplicity. Its advantages are durable retention, repeated reads by independent consumers, and a mature ecosystem for analytics and downstream integration. The disadvantages are the extra operational footprint of a broker cluster, partition-local rather than global ordering, and a non-trivial tuning burden around acknowledgements, partitioning, batching, and replication. Recent review work also notes that Kafka benchmarks are frequently hard to reproduce because configuration details are underreported, which strengthens the case for a carefully controlled thesis benchmark [11], [23], [24].

C. RabbitMQ

Earlier comparative work positions RabbitMQ as a routing-oriented messaging system that offers flexible exchanges, acknowledgements, and strong support for enterprise-style asynchronous messaging [23], [25]. Current RabbitMQ documentation additionally introduces *streams* as an append-only, non-destructive, always persistent and replicated data structure that complements traditional queues rather than replacing them. Stream consumers can attach at arbitrary offsets or timestamps, replay data, and scale out via super streams when partitioning is needed [26].

This makes RabbitMQ attractive as a hybrid comparison point between classic broker semantics and newer log-like

stream semantics within the same ecosystem. Its advantages are flexible routing, explicit acknowledgements, and a gradual migration path from queue-oriented messaging to append-only streams. The limitations are that classic RabbitMQ semantics are not optimised for replay-heavy logs, while stream mode introduces its own replication and partitioning trade-offs. RabbitMQ should therefore be evaluated not only on raw throughput, but also on how well it balances routing flexibility, operational continuity, and stream-specific behaviour [23], [25], [26].

D. NATS with JetStream persistence

The NATS ecosystem deserves separate consideration because it targets lightweight, location-transparent distributed communication. Official NATS documentation presents core NATS as a connective technology for modern distributed systems with publish/subscribe and request/reply communication, while JetStream adds persistence, replay, replication, and consumer state on top of the base message bus [27]–[29]. JetStream explicitly supports replay from the beginning of a stream, from a specific sequence number, or from a time-based position, and it offers both push-based and pull-based consumers for different workload profiles [29].

For the thesis, NATS is interesting because it represents a lightweight alternative to Kafka and RabbitMQ rather than a direct substitute for either. JetStream supports durable streams, replay, replication, and scalable pull consumers, while core NATS preserves a simple request/reply and pub/sub model that is attractive for microservice systems [28], [29]. A key complication is that comparative literature still often discusses the older NATS Streaming layer rather than JetStream, for example Sharvari and Sowmya Nag [25], which means that published results do not always map perfectly to the current product. NATS with JetStream is most promising when lightweight deployment, elastic scaling, and a smaller operational surface are valued more highly than Kafka’s larger ecosystem and benchmarking history.

E. Benchmark Design

1) *Methodological positioning and scope:* The benchmark design adapts recurring evaluation principles to the specific constraints of DYNAMOS rather than copying a single prior protocol. The controlled multi-stage pipeline follows the style of prior streaming-pipeline evaluations such as DataXc [7]; identical business logic across mechanisms follows comparative benchmarking practice for distributed systems, where Karimov et al. require the same workload definitions and measurement points across systems under test, and Henning and Hasselbring define scalability as the rate of throughput growth per added resource unit [8], [9]; the emphasis on throughput, latency, concurrency, resource pressure, and recovery reflects recurring evaluation dimensions in streaming and IPC (Inter-process communication) studies [10], [11].

The aim is not to identify a universally best mechanism, but to narrow the design space for DYNAMOS by asking which communication model best fits large, ordered, and

concurrent inter-service transfer under realistic operational constraints. The completed corpus evaluates five transport mappings against the same pipeline, the same payload definitions, and the same correctness criteria, and the results are interpreted in terms of DYNAMOS integration. After the DYNAMOS implementation showed that chunked unary transfer is a realistic design option, the benchmark harness was extended with a sixth mapping, batched unary gRPC, in which one unary request carries multiple logical records. The first repetition of that extended rerun campaign completed during the concept phase and is reported in Section III-F4; the completed five-transport corpus remains the basis for the main comparison tables. The chapter body keeps the main comparison compact: the completed single-node clean matrix, three focused operational scenarios, and the resulting architectural interpretation. Full preset definitions and detailed per-case tables remain in the appendix.

2) *Common testbed and fairness constraints:* All candidate mechanisms were evaluated in the same Go-based microservice chain deployed on Kubernetes: *Producer* → *Transformer* → *Sink*. The services ran in a dedicated namespace using the same manifests, the same payload schema, and the same transformation logic. Resource budgets were controlled through Kustomize overlays so that the comparison remained centered on the communication layer rather than on application-level differences [8], [9].

The transport mappings were also kept as close as possible to one another. Client-streaming and per-message unary gRPC provided the direct, service-to-service reference points in the completed corpus. Kafka preserved per-flow ordering through keyed partitioning. RabbitMQ used streams rather than classic queues. NATS used JetStream with asynchronous publishing and explicit consumer acknowledgements. The brokered mappings were tuned for batched or asynchronous sending rather than for one blocking publish acknowledgement per record, so the comparison remained centered on transport behaviour rather than on intentionally slow acknowledgement patterns. The new batched-unary mapping keeps the same logical messages as the other transports, but groups them into bounded unary requests so that the sink still measures logical records rather than RPC calls.

The service logic remained intentionally lightweight. The producer emitted synthetic or replayed records, the transformer applied a deterministic operation, and the sink recorded latency, pacing, and correctness summaries. That kept the benchmark focused on transfer behaviour, buffering, and recovery rather than on domain-specific computation [7], [10].

3) *Evaluation dimensions:* The literature consistently treats streaming evaluation as a multi-dimensional problem rather than a single throughput race [7], [10], [11]. This study follows that structure. Five dimensions matter most here: transfer efficiency, latency and pacing stability, overload behaviour, recovery correctness, and operational cost. A transport that leads the clean matrix but degrades badly under backpressure or restart is not automatically the best fit for DYNAMOS. The benchmark therefore keeps those dimensions visible through-

out the chapter.

4) *Scenario set and scenario-to-property mapping*: The completed RQ1 study uses four scenario families: a clean bulk-transfer matrix, a steady-rate continuous scenario, a slow-consumer backpressure scenario, and a forced-restart recovery scenario. This scenario set is not presented as a new benchmark methodology. It adapts recurring concerns from stream-processing and microservice evaluation literature, namely sustained transfer, latency stability, overload response, recovery after interruption, and correctness under at-least-once delivery [8], [10], [11], [30]. These properties are relevant to DYNAMOS because generated workflows may transfer large intermediate data, may contain stages with unequal processing rates, and may be restarted or redeployed as part of normal Kubernetes operation.

5) *Experimental factors*: The main independent variables are payload size, concurrency, target rate where applicable, sink slowdown in the backpressure case, and explicit failure injection in the recovery case. Concurrency denotes the number of parallel logical workers inside one benchmark run, not the number of producer Kubernetes jobs. In brokered modes, corresponding transformer and sink workers consume the worker-specific streams, subjects, or partitions. Resource budgets are held constant through Kubernetes overlays so that performance differences reflect the transport mapping rather than an uneven infrastructure allocation. This keeps the experiment matrix broad enough to expose real tradeoffs without turning the chapter into an exhaustive lab log [11], [25].

6) *Result presentation and appendix boundary*: The main text reports one clean-summary table, one compact size-wise clean breakdown, and three focused scenario tables for continuous flow, backpressure, and recovery. Detailed per-case clean-matrix tables remain in Appendix B. Resource metrics are used only where the sampling scope is the same across the compared cells. Broker-specific resource traces from exploratory runs are kept as supporting context rather than as a primary ranking signal. The RQ1 clean matrix should therefore be read as a screening study for candidate communication mechanisms; the DYNAMOS-in-situ evidence in RQ3 is the stronger evidence for the implemented chunked paths.

F. Results

1) *Metric definitions*: The benchmark reports the following metrics. The clean, continuous, and backpressure tables use the first group of metrics. The recovery table adds a second group because restart behaviour cannot be described by one number.

- **Message throughput**. Message throughput reports how many messages reach the sink per second. It is the main clean-matrix metric because it shows how much protocol and broker overhead each transport adds for the same logical workload.
- **Data throughput**. Data throughput reports the same transfer rate in MiB/s. It is included because two transports can move a similar number of messages while

carrying different effective byte volumes once framing, batching, and message size are considered.

- **Median latency**. Median latency is the typical end-to-end time between producer emission and sink observation. It helps separate transports that are usually fast from transports that only look good in aggregate throughput.
- **p_{95} and p_{99} latency**. These percentiles report tail latency. They matter because broker queues, batching, and consumer polling can keep average throughput high while delaying the slowest messages.
- **p_{95} inter-arrival**. Inter-arrival delay measures the spacing between consecutive messages at the sink. In the continuous and backpressure scenarios it shows whether the sink receives a regular flow or bursty delivery.
- **Jitter**. Jitter summarizes variation in inter-arrival timing. A low-jitter transport delivers messages in a more predictable rhythm, which is useful when downstream services depend on steady pacing.
- **Duplicates**. Duplicate counts show how often the sink observes the same logical message more than once. This is important for at-least-once systems because replay, retries, or commit timing can preserve delivery while still requiring idempotent downstream processing. Sink throughput is computed from unique logical messages; duplicate deliveries are counted separately and do not inflate the reported message rate.
- **Ordering violations**. Ordering violations count messages that arrive out of the expected per-flow sequence. This matters for DYNAMOS-style pipelines because some transformations assume that chunks from the same logical stream remain ordered.
- **Pre-failure throughput**. In the recovery scenario, pre-failure throughput confirms whether the transport was near the configured target before the restart. Without this value, a recovery result could be misleading because a transport might appear stable only because it was already under-producing.
- **Recovery-window throughput**. Recovery-window throughput measures delivery during the fixed window immediately after the forced restart. It shows how much useful progress continues while the system is reconnecting or replaying.
- **Post-recovery throughput**. Post-recovery throughput measures the rate after the immediate disruption has passed. It shows whether the transport returns to normal transfer behaviour or remains degraded after the restart.
- **First post-failure message**. This is the time until the first message is observed after the restart. It measures visible resumption, but not full recovery, since a transport can emit one message quickly while still carrying a large backlog.
- **Sustained 80%/5s recovery**. This is the first point at which the sink observes five complete one-second buckets at or above 80% of the target rate. It is stricter than first-message recovery because it requires stable progress rather than a single successful delivery.

Table I
SCENARIO-TO-PROPERTY MAPPING USED IN THE RQ1 BENCHMARK.

Scenario	Primary property	Why it matters for DYNAMOS
Clean bulk transfer	Protocol overhead, sustained throughput, multi-hop transfer efficiency	Large dataset exchange should remain efficient when data moves incrementally through a composed service chain.
Continuous steady-rate flow	Latency stability, inter-arrival regularity, pacing precision	Long-lived flows matter when downstream services depend on a predictable delivery rhythm.
Slow-consumer backpressure	Queue build-up, overload propagation, throttling versus delay accumulation	DYNAMOS pipelines can contain stages with unequal service rates, so overload handling must remain interpretable.
Forced-restart recovery	Reconnect behaviour, duplicates, ordering preservation, recovery time	Ephemeral microservices and restarts are normal in Kubernetes-style deployments.

- **Recovery debt.** Debt is the number of messages missing from the fixed recovery window relative to the configured target. It represents how much work the transport failed to deliver on time and therefore how much backlog or lost progress remains after disruption.
- **Recovery p_{95} latency.** This is the tail latency of messages observed in the recovery window. It reveals whether recovery is happening through smooth resumed delivery or through delayed backlog bursts.
- **Post-failure duplicates and ordering violations.** These are correctness counts restricted to the part of the run after the restart. Separating them from clean-run correctness makes it visible whether failures trigger replay side effects.
- **Backlog drain and completion.** Backlog drain measures how long it takes for delayed work to clear after the restart. The completion flag records whether the run eventually reached the expected message count.
- **Resource pressure.** CPU and memory measurements are used as supporting evidence where the same sampling scope is available across the compared runs. They indicate whether a transport improves throughput by adding acceptable overhead or by shifting pressure to the broker layer.

2) *Comparison scope:* The primary completed comparison in this chapter is a single-node Kubernetes benchmark across five transports: client-streaming gRPC, per-message unary gRPC, RabbitMQ Streams, NATS JetStream, and Kafka. Single-node means that the pipeline services and broker components are scheduled on one Kubernetes node, which isolates transport overhead without adding cross-node replication or placement effects. Kafka, RabbitMQ Streams, and NATS JetStream can also be deployed as multi-node broker clusters, where partitioning, replication, and placement can change both throughput and resilience [6], [22], [26], [29]. Exploratory clustered broker runs were performed, but they are not used as primary evidence because they expand the experiment from DYNAMOS communication design into broker-cluster tuning. The extended rerun campaign, which adds batched-unary gRPC and a 256 KiB payload tier, is reported separately in Section III-F4.

Table II shows a clear separation between the two direct gRPC mappings and the three brokered alternatives in the completed corpus. Client-streaming is the fastest transport in the clean matrix by a wide margin because it keeps one long-lived RPC open and sends many messages through that stream, avoiding the setup and response path of one unary call per message. Per-message unary is substantially slower because every message is transferred as its own request-response interaction, but the same blocking behaviour also keeps queueing low and explains its very low tail latency. This unary column is a lower-bound mapping, not the chunked unary design that is later implemented in DYNAMOS. The brokered transports form a middle band: they improve throughput over per-message unary by decoupling senders and receivers and by batching broker interaction, but they add broker persistence, routing, consumer polling, and backlog effects, which raises tail latency. Among the brokered transports, NATS JetStream achieves the highest average throughput, followed by RabbitMQ Streams and Kafka. Ordering violations are absent in the clean matrix. The duplicate counts on NATS and, to a much smaller extent, Kafka should be interpreted as at-least-once redelivery under explicit acknowledgements. The NATS consumer uses explicit acknowledgements with a configured `30 s AckWait`; if processing or acknowledgement falls behind that window, JetStream can redeliver a message. These duplicates do not inflate the throughput number because the sink increments message throughput only for first-seen sequence numbers.

3) *Clean bulk-transfer matrix:* The size-wise breakdown in Table III shows how the clean-matrix ranking changes with message size. Client-streaming remains the fastest option at every size. Its advantage over per-message unary is largest at small payloads, where per-request overhead dominates, and narrows as the payload grows. NATS JetStream is the strongest brokered transport across the four reported size bands. RabbitMQ Streams is competitive at 256 B and 1 KiB, then declines more sharply at larger payloads. Kafka stays below NATS in every size tier, but remains broadly stable and avoids the sharp collapse seen in RabbitMQ at 16 KiB.

All clean-matrix runs reached the target message count. Full per-case tables remain in Appendix B, but the main pattern is

Table II
SINGLE-NODE SYNTHETIC-CLEAN SUMMARY FROM THE COMPLETED FIVE-TRANSPORT CORPUS, AVERAGED ACROSS ALL PAYLOAD SIZES AND CONCURRENCY LEVELS.

Transport mode	Avg. msg/s	Avg. MiB/s	Avg. p_{95}	Avg. p_{99}	Duplicates	Ordering viol.
Client-streaming	31,259.76	51.35	242.82	441.59	0	0
Unary	2,291.37	9.55	3.01	31.23	0	0
RabbitMQ Streams	9,110.73	16.29	2,123.56	2,303.05	0	0
NATS JetStream	13,116.27	32.13	1,581.11	1,659.99	5,086	0
Kafka	7,354.42	20.12	1,729.34	1,901.42	9	0

Table III
SINGLE-NODE SYNTHETIC BULK-TRANSFER THROUGHPUT BY MESSAGE SIZE, AVERAGED ACROSS CONCURRENCY LEVELS 1, 4, 8, AND 16.

Msg size	Stream msg/s	Unary msg/s	RabbitMQ msg/s	NATS msg/s	Kafka msg/s
256 B	75,604	2,586	13,065	22,561	12,467
1 KiB	27,682	2,599	14,381	15,432	7,573
4 KiB	15,358	2,319	7,905	10,086	6,527
16 KiB	6,395	1,662	1,092	4,387	2,850

already visible here: client-streaming dominates bulk transfer, per-message unary remains the low-latency reference rather than a throughput candidate, and the three brokered transports form a separate performance band with NATS at its front. The concurrency levels in this table represent parallel logical workers within one benchmark run, so they should be read as intra-run parallelism rather than as multiple independent producer deployments. Several per-case cells are visibly non-monotonic, especially for RabbitMQ Streams at 1 KiB and 16 KiB and for NATS at 256 B. Those cells are treated as motivation for the three-repetition rerun campaign rather than as stable fine-grained rankings. The robust conclusion from the current corpus is the broad separation between direct streaming, brokered transfer, and per-message unary, not the exact ordering of every broker at every concurrency.

4) *Extended rerun: batched unary and the 256 KiB tier:* The extended rerun campaign adds the batched-unary mapping (100 records per unary request) and the 256 KiB payload tier that approximates the measured 273.7 KiB DYNAMOS chunk. Table IV reports the first completed repetition. The rerun executed on a freshly provisioned single-node cluster, so absolute throughput differs from the completed corpus (client-streaming, for example, is roughly twice as fast at 1 KiB); comparisons should be made within this table rather than across tables.

Two results matter for the thesis argument. First, batched unary closes most of the gap between per-message unary and client-streaming: at 1 KiB and concurrency 8 it reaches 61,184 msg/s against 55,478 for client-streaming and 3,596 for per-message unary in the same run, and it matches or exceeds

Table IV
EXTENDED RERUN, FIRST REPETITION: SINGLE-NODE SYNTHETIC-CLEAN THROUGHPUT (MSG/S) BY MESSAGE SIZE, AVERAGED ACROSS CONCURRENCY 1, 4, 8, AND 16. BATCHED UNARY CARRIES 100 RECORDS PER REQUEST. THE NATS 256 KiB CELL IS CONCURRENCY 1 ONLY; DASHES MARK CELLS THAT HAD NOT COMPLETED WHEN THIS VERSION WAS FINALIZED.

Msg size	Stream msg/s	Unary msg/s	Batched unary msg/s	RabbitMQ msg/s	NATS msg/s	Kafka msg/s
256 B	90,897	3,178	86,605	19,690	19,632	12,480
1 KiB	58,778	2,913	58,310	18,041	16,474	–
4 KiB	23,324	2,883	33,107	9,005	10,964	–
16 KiB	11,653	2,265	10,857	3,858	5,313	–
256 KiB	705	490	852	376	203	–

client-streaming at 4 KiB and 256 KiB. The per-message unary column should therefore be read as a lower bound on request-response designs: once per-request overhead is amortised over a batch, a unary-based mapping is competitive with direct streaming. This is exactly the mechanism that the chunked-unary DYNAMOS variant exploits in RQ3. Second, at the 256 KiB tier the gRPC mappings clearly outperform the brokered transports in data throughput: batched unary moves 212.9 MiB/s and client-streaming 176.3 MiB/s, against 94.0 MiB/s for RabbitMQ Streams. The brokered band’s competitiveness at small payloads does not extend to the message sizes that the DYNAMOS bulk workload actually moves.

The rerun also produced a finding of its own. The NATS JetStream mapping initially failed every 256 KiB cell: the harness had configured each stream with a 512 MiB retention cap, while one 256 KiB run moves roughly 5 GiB, so JetStream’s limits-based retention silently discarded unconsumed messages and the sink could never complete. After raising the cap, the concurrency-1 cell completed at 203 msg/s; the higher-concurrency NATS cells and the remaining Kafka tiers had not completed when this version was finalized and are reported as missing data rather than as transport results. The episode is itself informative: at realistic chunk sizes, broker retention limits become an operational failure mode that does not exist for the direct gRPC mappings. The rerun further confirms the duplicate accounting described earlier: NATS again produced tens of thousands of AckWait-driven

Table V

CONTINUOUS STEADY-RATE SCENARIO: SINGLE-NODE RESULTS (1 KiB PAYLOAD, 1,000 MSG/S TARGET, ONE REPETITION PER CELL). THROUGHPUT IN MSG/S; LATENCY, INTER-ARRIVAL, AND JITTER VALUES IN MILLISECONDS.

Transport mode	Concur.	Throughput	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	4	1,049.69	7.16	1.66	0.68
Client-streaming	8	1,050.93	7.14	1.65	0.70
Unary	4	1,050.84	0.88	2.10	1.09
Unary	8	1,050.73	0.91	2.11	0.99
RabbitMQ Streams	4	1,047.83	34.07	2.89	1.30
RabbitMQ Streams	8	1,049.16	49.36	3.51	2.11
NATS JetStream	4	1,052.71	26.67	1.95	0.84
NATS JetStream	8	1,052.03	102.25	3.07	2.71
Kafka	4	1,042.55	326.52	0.01	8.22
Kafka	8	1,033.59	387.36	0.03	7.94

redeliveries in otherwise clean runs, none of which inflate the reported unique-message throughput, while all other transports completed with zero duplicates and zero ordering violations.

Scenario evidence for batched unary from the same campaign is consistent with the clean matrix. It holds the continuous 1,000 msg/s target (1,051 msg/s observed), and across three recovery repetitions it sustains 1,937–1,980 msg/s with zero post-failure duplicates, in clear contrast to the client-streaming replay behaviour in Table VII. All rerun cells are single repetitions except recovery, so these values are first evidence rather than settled characterisation.

5) *Continuous steady-rate scenario*: The continuous scenario uses 1 KiB messages and a target rate of 1,000 msg/s. The purpose is not to maximize throughput, but to see how steadily each transport sustains an ongoing flow.

All five transports sustain the 1,000 msg/s target and overshoot it slightly (1,033–1,053 msg/s) in this run, so the comparison is dominated by latency stability rather than by who can keep up. Client-streaming and unary have by far the lowest tail latency, with unary p_{95} below 1 ms and client-streaming p_{95} around 7 ms in both concurrency settings. Among the brokered transports, NATS at concurrency 4 (26.7 ms p_{95}) and RabbitMQ Streams (34–49 ms p_{95}) are close to one another, while NATS at concurrency 8 widens to 102 ms. Kafka remains the slowest in the tail, with p_{95} between 327 ms and 387 ms; its inter-arrival distribution is dominated by short bursts (0.01–0.03 ms p_{95}) and its jitter is the highest of any transport (around 8 ms), reflecting Kafka’s batched delivery pattern.

6) *Backpressure scenario*: The backpressure scenario keeps the payload at 1 KiB and concurrency at 8, but adds a 2 ms sink delay while the producer targets 4,000 msg/s. This exposes how each transport behaves once downstream processing becomes the bottleneck.

Client-streaming, NATS, and RabbitMQ Streams all keep their realised throughput in the 2.5–3.3 kmsg/s range under pressure, well below the 4,000 msg/s target but clearly above the unary baseline. They achieve this by absorbing queueing delay: medians around 1.4–2.6 s and p_{95} values between 1.9 s

Table VI

BACKPRESSURE SCENARIO: SINGLE-NODE RESULTS (1 KiB, CONCURRENCY 8, 4,000 MSG/S TARGET, 2 MS SINK DELAY, ONE REPETITION PER TRANSPORT). THROUGHPUT IN MSG/S; LATENCY, INTER-ARRIVAL, AND JITTER VALUES IN MILLISECONDS.

Transport mode	Throughput	Median lat. p_{95}	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	3,333.93	1,443.26	1,878.90	2.14	0.64
Unary	2,091.96	0.53	1.41	3.40	1.23
RabbitMQ Streams	2,557.86	2,603.70	3,476.80	2.26	1.68
NATS JetStream	2,973.05	1,768.99	3,034.92	2.14	0.69
Kafka	1,268.34	4,879.51	11,363.59	2.53	1.32

and 3.5 s. Unary gives up throughput earlier (2,092 msg/s) and therefore stays extremely low-latency for the requests that complete (median 0.5 ms, p_{95} 1.4 ms), which is the expected effect of per-request acknowledgement under sink slowdown. Kafka is the most throughput-constrained transport in this single-node backpressure case (1,268 msg/s) and develops the heaviest tail of any transport (p_{95} above 11 s), again reflecting its batched delivery pattern under load.

7) *Recovery scenario*: The recovery scenario uses 4 KiB messages at 2,000 msg/s and forces a transformer restart after 3 seconds. The table separates the run into pre-failure throughput, a fixed 10 s recovery window, and the post-recovery tail so that restart visibility, sustained recovery, backlog drain, and correctness cost are not collapsed into one ambiguous number.

Before failure, all transports are close enough to the configured 2,000 msg/s target to make the recovery comparison meaningful. The disruption then separates the mechanisms sharply. Client-streaming produces the fastest visible resumption, with the first post-failure message arriving within 1 ms, but its recovery-window throughput falls to 939 msg/s and it accumulates 10,630 messages of throughput debt. Its 10,615 post-failure duplicates are a direct consequence of the current reconnect policy: after a stream send fails, the producer reopens the stream and replays the worker stream rather than resuming exactly at the failed chunk. The historical 20,000-message run ends before five complete stable buckets can be observed after the later recovery point, so the sustained-recovery field is intentionally blank rather than inferred.

Unary shows the opposite shape. It preserves correctness, but it delivers no messages in the fixed recovery window, waits roughly 21.8 s before the first post-failure message, and accumulates the full 20,000-message recovery-window debt. It eventually records a sustained five-second window at 22 s, but by then the useful distinction is already visible: unary turns the restart into an extended outage rather than a buffered drain.

The brokered transports expose three different buffering behaviours. RabbitMQ Streams resumes within 293 ms and drains the run fastest, with recovery-window throughput slightly above target, but it pays for that with 886 post-failure duplicates and a recovery-window p_{95} latency near 3.9 s. Its post-recovery cell is empty because the run drains completely inside the fixed recovery window, leaving no post-window traffic to measure. NATS JetStream delivers its first post-failure

Table VII

RECOVERY SCENARIO: SINGLE-NODE RESULTS (4 KiB, CONCURRENCY 8, 2,000 MSG/S, TRANSFORMER RESTART AT 3 S). ONE REPETITION PER TRANSPORT. THROUGHPUT VALUES ARE IN MSG/S; TIME, LATENCY, AND DRAIN VALUES ARE IN MILLISECONDS. SUSTAINED RECOVERY IS THE FIRST FIVE COMPLETE ONE-SECOND BUCKETS INSIDE THE CONFIGURED TARGET BAND, WITH NO FALLBACK TO BACKLOG DRAIN. DASHES INDICATE THAT THE VALUE WAS NOT OBSERVED IN THE RUN OR THAT NO MESSAGES ARRIVED IN THAT PHASE/WINDOW.

Transport mode	Pre msg/s	Recovery msg/s	Post msg/s	First post-fail	Sustained 80%/5s	Debt	Recovery p_{95}	Post-fail dup.	Post-fail order	Backlog drain	Done
Client-streaming	2,711	939	1,999	0.18	–	10,630	2.74	10,615	0	11,446.21	Yes
Unary	2,161	0	777	21,824.57	22,000	20,000	–	0	0	27,734.03	Yes
RabbitMQ Streams	1,778	2,011	–	292.74	–	4,868	3,921.74	886	0	7,463.72	Yes
NATS JetStream	2,297	1,351	0.10	2.56	–	7,847	388.89	0	2	29,228.03	Yes
Kafka	2,343	0	710	29,224.26	–	20,000	–	62	0	29,760.29	Yes

message after only 2.6 ms, but the fixed recovery window still shows 7,847 messages of debt and no sustained recovery because the early replay plateau is followed by an extended quiet period; that quiet period is what appears as the near-zero post-window rate in the table. Kafka is the most delayed in this run: no messages arrive during the recovery window, the first post-failure message appears after about 29.2 s, and the run remains backlog-dominated until completion, although it keeps post-failure duplicates low and preserves ordering.

8) *Discussion*: The benchmark and the DYNAMOS architecture do not point to one simple answer, and that distinction matters. In the isolated pipeline benchmark, client-streaming gRPC is the strongest raw transport: it leads the clean matrix, remains stable in the continuous scenario, and avoids the broker overheads seen in the other transports. At the same time, the per-message unary result should not be read as proof that request-response transfer is unusable. It is a lower-bound mapping that pays one RPC per logical message. The DYNAMOS implementation later tests the more realistic request-response alternative: chunked unary transfer, where one unary message carries a bounded batch of rows. The extended rerun already supports that design directly: its batched-unary mapping closes most of the client-streaming versus unary gap and leads all transports at the 256 KiB tier that matches the measured DYNAMOS chunk size (Section III-F4).

DYNAMOS as implemented today has two different communication regimes. Within a deployed microservice-chain job, services communicate locally over gRPC inside a generated execution unit, where direct streaming is a natural optimisation. Across agents, DYNAMOS relies on sidecar-mediated RabbitMQ communication, ephemeral per-job queue management, and retry logic that tolerates the destination job not being ready when data is sent. The main lesson from RQ1 is therefore not that one named broker should be selected immediately. It is that the implementation should preserve chunks at the existing DYNAMOS boundaries and then decide separately whether a direct stream or a brokered stream is the better boundary mechanism.

This reframes the implementation hypotheses used in RQ2. The primary hypothesis is chunked transfer at existing boundaries, because that attacks per-message overhead and full-result

buffering without discarding the DYNAMOS execution model; the rerun’s batched-unary results now back this hypothesis with transport-level measurements rather than inference alone. RabbitMQ Streams is a secondary hypothesis for the distributed agent boundary. It preserves temporal decoupling, offers retained records and replay, and fits the current RabbitMQ-oriented operational model, but the local benchmark does not prove that it is faster than ordinary chunked transfer. Client-streaming gRPC remains the raw-efficiency baseline and a strong candidate for pod-local service chains, but it would need explicit reconnect, replay, and idempotence logic at DYNAMOS boundaries.

Kafka and NATS JetStream are lower-priority options for the current DYNAMOS architecture. Both would require a more substantial platform change, and both bring strengths, large-scale replay, log-centric consumption, multi-consumer stream processing, or subject-based pub/sub flexibility, that DYNAMOS’ present request-shaped SQL workloads do not strongly exploit. They remain valuable comparison points in the benchmark and possible future directions if DYNAMOS evolves toward persistent, replay-heavy, or multi-consumer workloads, but they do not appear to be the most pragmatic next step for the system as it exists today.

RQ2 therefore uses RQ1 as a design-space filter rather than as a final implementation verdict. RQ3 then validates the chunk-aware paths inside DYNAMOS, where startup behaviour, policy enforcement, failure handling, and real workload structure can be weighed against the isolated transport-level results reported here.

G. Threats to Validity

Four limits are important when interpreting these results. First, the primary five-transport comparison is based on completed single-node runs, and the focused operational scenarios are point measurements rather than confidence-interval estimates. They are strong enough for comparative interpretation, but not for a fully statistical treatment. Second, several clean-matrix cells show non-monotonic behaviour, especially in the brokered modes. The broad ranking is useful, but the exact within-broker ordering should be revisited with the three-repetition rerun campaign. Third, the completed cor-

pus tops out at 16 KiB payloads, while a measured 5,000-row DYNAMOS chunk is approximately 280,255 bytes as a protobuf `MicroserviceCommunication`. The rerun’s 256 KiB tier closes this gap for five of the six transports, but currently at a single repetition, on a freshly provisioned cluster whose absolute throughput differs from the completed corpus, and with the Kafka tiers and higher-concurrency NATS cells still incomplete. Fourth, resource measurements are supporting evidence rather than the basis for the transport ranking. They are reported only where the same sampling scope is available across compared runs, because otherwise a broker-heavy transport could appear more expensive simply because more of its infrastructure was included in the measurement.

IV. STREAMING INTEGRATION IN DYNAMOS

This section addresses RQ2 by moving from a generic transport benchmark to an implementation inside DYNAMOS. The question is no longer only which communication mechanism is fast. The implementation must also preserve DYNAMOS’ policy approval path, generated job model, sidecar-mediated communication, and isolation between data stewards.

A. Motivation

The original DYNAMOS implementation already identifies large message transfer as a limitation. The DYNAMOS thesis notes that the default maximum size of a gRPC method is around 4 MB, corresponding to approximately 30,000 records in the original experiments, and that the current gRPC implementation uses unary RPCs: one request and one response [1]. The same discussion observes that increasing the message size is theoretically possible but memory-heavy, and explicitly names streaming as the unfinished solution. This makes large-result transfer an appropriate point at which to evaluate streaming techniques inside DYNAMOS rather than only in a synthetic pipeline.

The implementation target is deliberately different from the average-query example commonly used in DYNAMOS demonstrations. An average query is a reduction workload: it may scan many records but returns one scalar result. That tests computation near the data source, but it does not test whether DYNAMOS can transport many aligned rows. Bulk retrieval therefore stresses the part of DYNAMOS that is most affected by full-response buffering. It is also a useful precursor to later distributed learning scenarios, but this thesis first validates whether the core DYNAMOS data path can move large row sets incrementally.

B. DYNAMOS execution path

DYNAMOS separates the control path from the data path. A client request first passes through the gateway, policy evaluation, orchestration, and the relevant agents. Only after that approval and composition step are generated job pods used to execute the request. For the `dataThroughTtp` archetype, the data provider job retrieves rows, the Trusted Third Party (TTP) or SURF-side job performs the compute-provider part

of the chain, and the response eventually returns through the agent and gateway path.

Figure 1 shows the boundary that matters for the implementation. In the original path, a large result can be rebuilt several times before the client sees anything: inside the SQL response, inside the generated service chain, inside the agent response, and again at the gateway. The streaming implementation therefore treats chunk preservation as the central requirement. A transport is only useful for this workload if the first rows can leave the provider path before the final rows have been read and serialized.

The implementation should not be a special case for one benchmark request. DYNAMOS supports multiple archetypes and multiple data providers, so a chunk-aware path must keep existing request shapes valid. The primary evaluation uses `dataThroughTtp` with one provider to isolate the large-result bottleneck. RQ3 then adds focused checks for `computeToData` and for `dataThroughTtp` with two providers, because those scenarios exercise different parts of the generated job and aggregation path.

C. Common chunk model

All chunked variants use the same logical stream model. The SQL query service reads the bulk result in fixed row batches. Each batch carries stream metadata: provider identity, correlation identifier, sequence number, row count, partial or final status, and ordered column names. The ordered-column metadata is important because Go protobuf structs can otherwise expose map fields in nondeterministic order, which would make equal results appear different across transports.

The aggregate and algorithm stages preserve chunk boundaries for non-average bulk workloads. They do not merge all incoming rows into one table unless the request explicitly uses the classic unary baseline. The gateway exposes chunk progress to the benchmark client through `NDJSON providerResult` events, with partial events before the final event. This keeps the external response shape compatible with the existing gateway while making progress visible.

Policy enforcement is not moved into the stream. DYNAMOS still evaluates the request before execution, constructs the permitted job path, and then lets that generated path process the approved data exchange. The current implementation therefore preserves the original policy decision point. It does not claim per-chunk policy revocation during a running stream.

D. Implemented variants

Four paths are compared in the thesis. The first is the original DYNAMOS style and the other three are chunk-aware implementations. This distinction matters because the RabbitMQ Streams path still needs chunking before it reaches the broker. If the generated job first built one giant result and only then wrote it into RabbitMQ Streams, the client would not receive an earlier first result and the memory problem would largely remain.

The classic unary baseline uses the original DYNAMOS main branch at commit `a2d9531` [31]. For the benchmark,

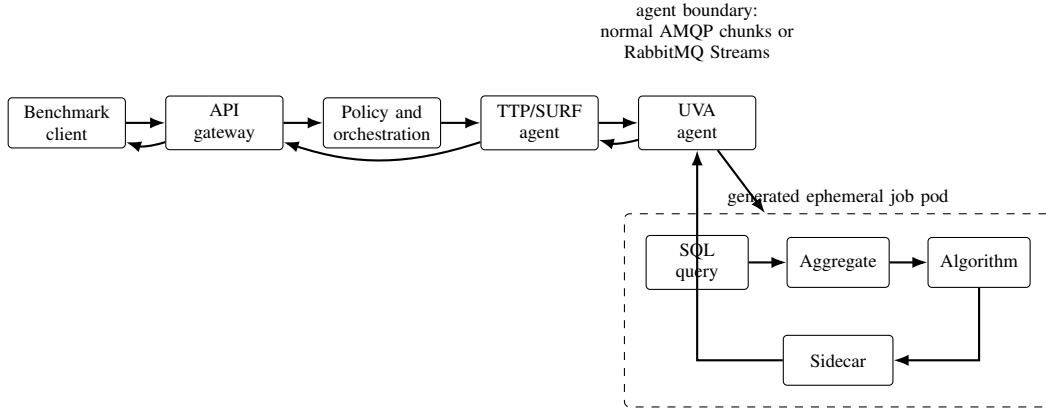


Figure 1. DYNAMOS bulk retrieval path used for RQ2. The implementation changes the bulk data path after policy approval. Chunking starts at the SQL query service and must be preserved through the generated job, agent path, and gateway response to avoid rebuilding one complete result table before the client sees progress.

Table VIII
DYNAMOS IMPLEMENTATION VARIANTS AND THE ARCHITECTURAL BOUNDARY CHANGED BY EACH VARIANT.

Variant	In-pod service chain	Agent boundary	Buffering behaviour	Architectural purpose
Classic unary	Unary gRPC, full response	Classic RabbitMQ message path	Complete result is buffered into one logical response	Baseline closest to original DYNAMOS once the gRPC size ceiling is raised.
Chunked unary	Repeated unary calls carrying row chunks	Classic RabbitMQ carries separate chunk messages	Chunks stay separate through aggregate, algorithm, agent, and gateway	Smallest architectural change; preserves the message-driven style.
Intra-job gRPC streaming	Long-lived <code>SendDataStream</code> through the generated service chain	Classic RabbitMQ carries separate chunk messages	Chunks stream inside the pod and remain chunks after leaving it	Tests native gRPC streaming where DYNAMOS already uses local gRPC.
RabbitMQ Streams at agent boundary	Chunked unary path inside the generated job	RabbitMQ stream records replace classic queue messages for chunk transfer	Local chunks are written as broker stream records and reassembled downstream	Tests stream-log semantics for distributed inter-agent transfer.

its gRPC and message-size limits are configured high enough to complete the tested row counts. This avoids treating the historical 4 MB ceiling as the result. Classic unary remains intentionally full-buffered, because that is the behaviour the streaming implementations are meant to improve. The multi-provider classic baseline also includes a minimal aggregation correction so the first provider is not appended twice during result merging. This correction makes the baseline result valid; it does not add streaming behaviour.

The three chunked paths use the streaming implementation branch at commit 923b018 [32]. Chunked unary is the least disruptive path. It keeps unary calls and the existing agent style, but it sends many smaller result messages instead of one large result. Intra-job gRPC streaming replaces repeated unary calls inside the generated service chain with one stream. That matches gRPC’s native streaming model [21], but it couples sender and receiver lifetimes more tightly. RabbitMQ Streams changes the agent boundary. It keeps the chunked local path, then uses RabbitMQ’s append-only stream abstraction for chunk transfer between agents [26].

These variants are not three independent full-stack streaming systems. They are interventions at different parts of the same DYNAMOS path. The local performance improvement

of the RabbitMQ Streams variant is therefore not caused by RabbitMQ Streams alone. It is caused by the common chunk-aware data path, with RabbitMQ Streams adding a different broker representation at the agent boundary.

E. RQ2 answer

Streaming can be integrated into DYNAMOS by changing the bulk data path after policy approval while leaving the control path intact. The implementation introduces an explicit chunk model at the SQL query service, preserves those chunks through aggregate and algorithm services, carries them through the agent boundary, and exposes partial result events to the client. This keeps dynamic service composition and policy-driven orchestration in place, while removing the need to repeatedly materialize one complete result table before progress is visible. Compatibility with `computeToData` and multi-provider `dataThroughTtp` is treated as part of the implementation requirement rather than as an optional benchmark extension.

The best architectural choice is boundary-specific. Chunked unary is the best current local default because it gives chunking without changing the DYNAMOS execution model very much. Intra-job gRPC streaming is useful where both endpoints are already in the generated pod and a direct stream is

acceptable. RabbitMQ Streams is more relevant at the inter-agent boundary, especially for future distributed deployments where retained streams, replay, and independent consumers may matter more than local latency.

V. EVALUATION OF DYNAMOS STREAMING IMPLEMENTATIONS

This section addresses RQ3 by validating the implemented DYNAMOS data paths on a large-result workload. RQ2 explains how the variants are integrated. RQ3 measures whether those variants improve completion time, time-to-first-result, and resource usage compared with the classic unary baseline.

A. Benchmark method

The primary benchmark uses the `dataThroughTtp` archetype with a single data provider, UVA. The request still follows the TTP/SURF compute-provider path defined by the archetype, but the data fanout is intentionally limited to one provider so that the experiment isolates large-result transfer rather than multi-provider coordination.

Each request executes a bulk projection query over the large `PersonenLarge` and `AanstellingenLarge` tables and joins them on `Unieknr`. The result is limited to 50,000, 100,000, or 250,000 rows. The chunked variants use 5,000 rows per SQL chunk, which gives 10, 20, and 50 total chunks respectively. This value is an experimental configuration rather than a claimed optimum: it is large enough to avoid turning the benchmark into a per-row messaging test and small enough to expose partial progress. In the saved 50k-row bulk output, one 5,000-row chunk serialized as a DYNAMOS protobuf `MicroserviceCommunication` is approximately 280,255 bytes, or 273.7 KiB. This explains why the RQ1 rerun campaign adds a 256 KiB synthetic payload tier.

Four implementations are compared. Classic unary is taken from the original DYNAMOS `main` branch at commit `a2d9531`. The chunked unary, intra-job gRPC streaming, and RabbitMQ Streams variants are taken from the streaming implementation branch at commit `923b018`. Every primary `dataThroughTtp` cell uses 10 warm repetitions. Runs are accepted only when row counts match the expected limit and canonical content hashes validate the returned content. For chunked variants, the benchmark also requires partial result events before completion.

Two focused scenario checks are added after the primary benchmark. The first changes the archetype to `computeToData`. The second keeps `dataThroughTtp` but uses two data providers, UVA, VU. These checks are included to show whether the implementation remains compatible outside the single-provider TTP path. For these concept-stage checks, the chunked variants use the completed three-repetition matrix, while the classic unary baseline was rerun with 10 repetitions after correcting the multi-provider aggregate merge.

Resource usage is sampled from Kubernetes once per second across the relevant DYNAMOS namespaces and summarized as peak total CPU millicores and peak total memory in

Table IX
MEDIAN COMPLETION LATENCY FOR DYNAMOS BULK RETRIEVAL. ALL CELLS COMPLETED 10/10 VALID REPETITIONS.

Implementation	50k rows	100k rows	250k rows
Classic unary	14.300s	19.501s	31.885s
Chunked unary	7.731s	7.870s	12.158s
Intra-job gRPC streaming	7.278s	9.299s	12.495s
RabbitMQ Streams boundary	7.809s	9.394s	14.756s

MiB. These sampled peaks are useful for comparing memory pressure, but they may underestimate short instantaneous spikes between samples. The measured request window starts immediately before the benchmark client submits the HTTP request and ends when the final response or final NDJSON event is observed. This means policy handling, orchestration, generated job creation, pod startup, data transfer, and response serialization are inside the measured window for both classic and chunked variants. The classic unary baseline required generated `surf`, `uva`, and `vu` jobs to be cleaned between repetitions. That cleanup happens before and after a measured request, not inside the request timing. Without it, later repetitions could return fast 502 or empty-provider responses because stale generated jobs were still present. The cleaned baseline is therefore the comparison used here.

B. Completion latency

Table IX and Figure 2 show that all three chunk-aware variants reduce median completion latency compared with classic unary. At 250k rows, chunked unary is 2.62x faster than classic unary, intra-job gRPC streaming is 2.55x faster, and the RabbitMQ Streams boundary variant is 2.16x faster. The 100k row case is especially favourable to chunked unary, which finishes in 7.870s compared with 19.501s for classic unary. The near-flat scaling of the chunked variants between 50k and 100k rows is explained by the fixed control-path cost analysed with Table XI: at these sizes most of the measured time is policy handling, orchestration, and job startup rather than transfer.

The median values hide some run-to-run spread, but the conclusion remains stable at the largest row count. At 250k rows, classic unary ranged from 29.615s to 50.854s. Chunked unary ranged from 10.985s to 23.233s, intra-job gRPC streaming from 10.566s to 23.311s, and RabbitMQ Streams from 12.724s to 26.520s. The long-tail runs show why the final thesis should use repeated runs rather than single measurements, but the gap between classic unary and the chunked paths is large enough that it does not depend on one unusually fast or slow run.

C. Additional scenario checks

Table X shows that the chunk-aware paths are not limited to the primary single-provider TTP case. For `computeToData`, the improvement is smaller because the request avoids part of the TTP aggregation path, but the chunked variants still complete faster than classic unary at both tested sizes. For multi-provider `dataThroughTtp`, the improvement remains large:

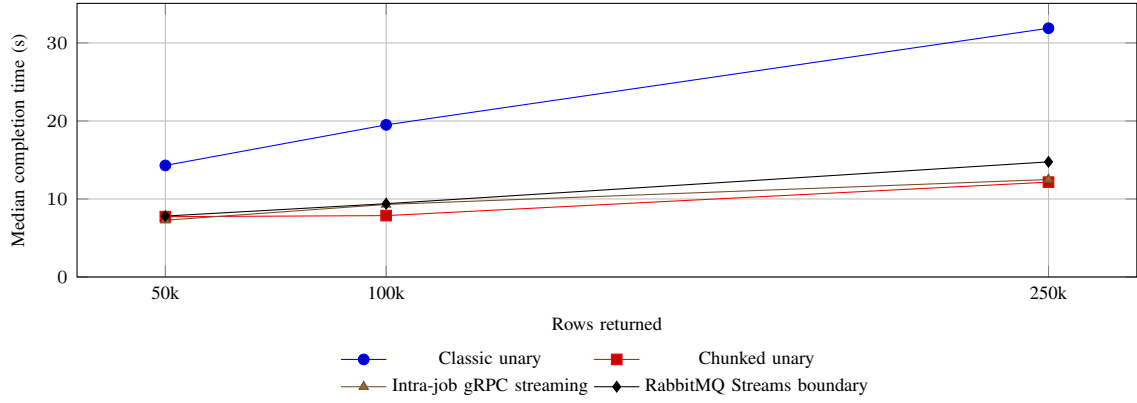


Figure 2. Median completion time for the local DYNAMOS bulk retrieval benchmark. Chunking is the dominant improvement: all chunk-aware variants scale better than classic unary as the returned row count grows.

Table X
FOCUSED DYNAMOS SCENARIO CHECKS BEYOND THE PRIMARY SINGLE-PROVIDER `DATAThroughTTP` BENCHMARK. THE EXTRA CHUNKED-VARIANT CELLS USE THREE REPETITIONS; THE CLASSIC CELLS USE THE CLEANED 10-REPETITION BASELINE.

Scenario	Providers	Rows per provider	Total rows	Classic unary	Chunked unary	Intra-job gRPC streaming	RabbitMQ Streams boundary
computeToData	UVA, VU	50k	100k	9.660s	5.584s	4.895s	5.371s
computeToData	UVA, VU	250k	500k	16.290s	10.913s	10.876s	13.140s
dataThroughTtp	UVA, VU	50k	100k	13.163s	5.820s	5.322s	7.226s
dataThroughTtp	UVA, VU	250k	500k	33.920s	13.444s	12.928s	14.883s

Table XI
TIME-TO-FIRST-RESULT AND PARTIAL RESULT BEHAVIOUR. CLASSIC UNARY HAS NO PARTIAL RESULTS, SO FIRST RESULT EQUALS COMPLETION.

Implementation	Rows	First result	Done	Post-first	Partial chunks
Classic unary	50k	14.300s	14.300s	0.000s	0
Classic unary	100k	19.501s	19.501s	0.000s	0
Classic unary	250k	31.885s	31.885s	0.000s	0
Chunked unary	50k	7.186s	7.731s	0.857s	9
Chunked unary	100k	6.054s	7.870s	1.734s	19
Chunked unary	250k	5.857s	12.158s	6.359s	49
Intra-job gRPC streaming	50k	6.522s	7.278s	0.605s	9
Intra-job gRPC streaming	100k	7.682s	9.299s	1.917s	19
Intra-job gRPC streaming	250k	6.459s	12.495s	6.128s	49
RabbitMQ Streams boundary	50k	6.915s	7.809s	0.819s	9
RabbitMQ Streams boundary	100k	6.737s	9.394s	2.611s	19
RabbitMQ Streams boundary	250k	7.014s	14.756s	7.861s	49

at 250k rows per provider, the classic baseline returns 500k total rows in 33.920s, while the chunked variants complete in 12.928s to 14.883s.

These scenario checks should be read as compatibility and direction-of-effect evidence rather than as the final statistical result set. They use fewer repetitions for the chunked variants than the primary benchmark. Their value for the concept thesis is that they show the implementation still works when the archetype and provider count change, and that the speedup is strongest when DYNAMOS must move large row sets through the TTP path.

D. Time to first result

Table XI shows the qualitative benefit of chunking. Classic unary cannot expose progress: the first result is the final result.

The chunked variants emit partial results substantially earlier. At 250k rows, the first partial result appears after about 5.9s to 7.0s, while completion follows after 12.2s to 14.8s. The post-first column estimates how much of the measured time remains after the first visible chunk. It is not a pure network-transfer measurement, because the first chunk still includes policy handling, job creation, and initial SQL execution. It is nevertheless useful because it separates request startup from the observable chunk-drain phase. Read together with Table IX, this decomposition shows that the end-to-end ratios understate the data-path difference: every chunked variant spends roughly 6–7 s before the first chunk regardless of row count, which is the shared policy, orchestration, and job-startup cost. Using that estimate for the control path, the 250k-row comparison becomes approximately 6.4 s of chunk drain for chunked unary against roughly 26 s of post-startup time for classic unary, a difference near $4\times$ where the end-to-end ratio reports $2.62\times$, because both end-to-end numbers include the same fixed control-path cost. End-to-end remains the headline metric since it is what a DYNAMOS client experiences. This decomposition also matters for workflows that can process data incrementally and for diagnosing whether a large request is still making progress.

E. Resource usage

The resource data in Table XII and Figure 3 support the main architectural claim. At 250k rows, classic unary reaches 3,695 MiB peak memory. Chunked unary stays at 1,378 MiB, intra-job gRPC streaming at 1,495 MiB, and the RabbitMQ

Table XII
PEAK SAMPLED RESOURCE USAGE DURING DYNAMOS BULK
RETRIEVAL. CPU IS PEAK TOTAL MILLICORES; MEMORY IS PEAK TOTAL
MiB ACROSS SAMPLED KUBERNETES NAMESPACES.

Implementation	Rows	Peak CPU	Peak memory
Classic unary	50k	782m	1,571 MiB
Classic unary	100k	813m	2,046 MiB
Classic unary	250k	1,198m	3,695 MiB
Chunked unary	50k	403m	1,151 MiB
Chunked unary	100k	469m	1,723 MiB
Chunked unary	250k	1,097m	1,378 MiB
Intra-job gRPC streaming	50k	351m	1,198 MiB
Intra-job gRPC streaming	100k	479m	1,230 MiB
Intra-job gRPC streaming	250k	1,418m	1,495 MiB
RabbitMQ Streams boundary	50k	457m	1,161 MiB
RabbitMQ Streams boundary	100k	982m	1,436 MiB
RabbitMQ Streams boundary	250k	1,687m	1,715 MiB

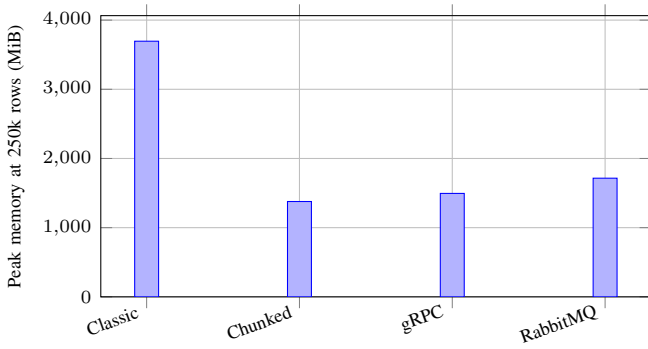


Figure 3. Peak sampled memory at 250k rows. Chunk-aware variants avoid the largest full-result buffer and keep memory below the classic unary baseline.

Streams boundary variant at 1,715 MiB. Chunking therefore reduces memory pressure consistently.

CPU is more nuanced. Chunked unary uses slightly less peak CPU than classic unary at 250k rows, but intra-job gRPC streaming and the RabbitMQ Streams boundary variant peak higher. This is expected. Chunking replaces one large buffer with more messages, more metadata handling, more serialization boundaries, and, in the RabbitMQ Streams variant, broker stream handling and reassembly. The defensible claim is therefore that chunking improves latency and memory. CPU remains transport-dependent.

F. Failure modes and robustness

Table XIII is included to avoid overstating robustness. The current benchmark supports a narrow robustness claim: chunking removes size-dependent single-message failure classes, reduces full-result memory pressure, and exposes partial progress before completion. It does not yet prove full recovery from arbitrary pod crashes, broker outages, or mid-stream policy revocation inside DYNAMOS. Those require explicit failure-injection tests in the DYNAMOS deployment, similar in spirit to the RQ1 recovery scenario.

G. Discussion and RQ3 answer

The DYNAMOS-specific benchmark changes the interpretation of the isolated RQ1 transport study. RQ1 showed that client-streaming gRPC is the strongest raw transport in the controlled pipeline. The DYNAMOS evaluation shows that integration fit matters as much as raw transport speed. DYNAMOS is a policy-driven, dynamically composed middleware in which decoupling, generated job lifecycle, sidecar communication, and diagnosable failure behaviour are part of the design problem.

The central RQ1 hypothesis partly survives validation, but in a more precise form. Chunking is the main source of the improvement, not the name of the transport alone. RabbitMQ Streams remains useful to study as a distributed-boundary mechanism, but in the local DYNAMOS runs it adds overhead compared with ordinary chunked transfer. This is why the RQ3 conclusion ranks chunked unary highest for the current local system while keeping RabbitMQ Streams as a future distributed candidate.

For the current local DYNAMOS implementation, chunked unary is the strongest default candidate. It has the best median completion time at 100k and 250k rows, the lowest 250k memory peak, and the smallest architectural change from the original message-driven design. It also shows why the main bottleneck was not simply “gRPC versus broker”, but full-result buffering across the DYNAMOS path.

Intra-job gRPC streaming remains a strong technical option. It is close to chunked unary at 250k rows and slightly faster at 50k rows. Its main weakness is architectural. Long-lived direct streams couple producer and consumer lifetime more tightly, which complicates dynamic composition, restart handling, and load balancing in Kubernetes-style deployments [3]. It is therefore best interpreted as a targeted optimisation for controlled in-pod or direct service-to-service flows.

The RabbitMQ Streams boundary variant should be interpreted carefully. It beats classic unary because it uses the common chunk-aware path, not because RabbitMQ Streams is faster than normal RabbitMQ chunks in the local benchmark. Compared with chunked unary, it is slower and uses more peak CPU and memory in this run. Its value is therefore not local latency. Its value is the stream-specific broker model: retained records, offset-based replay, non-destructive consumption, and a closer fit for distributed inter-agent transfer. Those properties are more relevant to Scattered Directive and FABRIC-style deployments than to a single local cluster.

The answer to RQ3 is that the implemented chunk-aware DYNAMOS data paths improve local bulk-retrieval scalability compared with classic unary. They reduce completion time, expose earlier partial results, and lower peak memory usage for 50k, 100k, and 250k row result sets. The best current local default is chunked unary. Intra-job gRPC streaming is a close alternative for pod-local service chains. RabbitMQ Streams should be kept as a distributed-boundary candidate, but its advantage must be validated under cross-site conditions rather than inferred from the local latency result.

Table XIII
FAILURE MODES OBSERVED OR ACCOUNTED FOR DURING THE DYNAMOS BENCHMARK WORK.

Failure mode	Affected path	Observation	Treatment	Robustness claim
Default gRPC message ceiling	Original classic unary	Original DYNAMOS documentation reports a practical limit around 4 MB, or roughly 30k rows in that setup	Increased benchmark message limits for the baseline so the comparison measures buffering cost rather than an artificial ceiling	Chunking removes the dependency on one very large response message
Full-result buffering	Classic unary and early bulk prototypes	Large results were materialized repeatedly before the client saw progress	Preserved 5k-row chunks through SQL, aggregate, algorithm, agent, and gateway stages	Supported by lower memory peaks and earlier partial results
Stale generated jobs	Repeated classic unary runs	Later repetitions could return fast 502 or empty-provider responses when old generated jobs remained	Cleaned generated <code>surf</code> , <code>uva</code> , and <code>vu</code> jobs outside the measured request window	Not a streaming advantage; handled for benchmark fairness
Multi-provider merge error	Classic baseline multi-provider	The first provider could be appended twice during aggregation	Applied a minimal aggregation correction before reporting multi-provider baseline timings	Baseline correctness fix, not a streaming feature
Timeout and empty-provider reporting	Early large-result runs	Some failures appeared as missing final results or empty provider bodies	Made timeout/error outcomes visible in benchmark output and separated clean errors from valid completions	Improves diagnosability; not a full crash-recovery proof

H. Threats to validity

The RQ3 evidence has five main limits. First, the benchmark runs on a local Kubernetes deployment, so it does not capture cross-site latency, bandwidth variation, or independent administrative domains. Second, the primary `dataThroughTtp` result uses 10 repetitions per cell, but the additional `computeToData` and multi-provider chunked checks currently use three repetitions. They support compatibility and direction of effect, not final statistical confidence. Third, resource usage is sampled once per second, so peak CPU and memory values can miss short spikes. Fourth, the fixed 5,000-row chunk size is justified experimentally but not optimized across all workloads. Fifth, the measured request window includes policy handling, orchestration, generated job startup, and response serialization. That is fair for comparing user-visible DYNAMOS behaviour, but it compresses the apparent transfer-only speedup because fixed control-path costs remain present in every implementation.

VI. FUTURE WORK

The local DYNAMOS benchmark is not the final distributed scalability answer. This thesis validates the local implementation first, because the basic streaming path must work before it is meaningful to test it across sites. The natural next step is Scattered Directive, which extends DYNAMOS with cross-node microservice chains for policy-aware VFL workflows and uses FABRIC to automate distributed deployment [33], [34].

In that setting, bulk transfer can become part of a larger VFL-style workflow. Parties may hold different features for overlapping entities, and a TTP or shared execution component may coordinate aligned intermediate values, labels, embeddings, model updates, or other training artefacts. The current thesis does not claim to validate a complete VFL training workload. It validates the DYNAMOS communication path that such a workload would depend on when large intermediate results have to cross generated services and agent boundaries.

FABRIC provides programmable compute and storage resources connected by high-speed dedicated links, which makes

it a suitable environment for testing DYNAMOS when the TTP and data providers are separated across sites [35], [36]. A later FABRIC experiment should keep the RQ3 metrics from this section and add network throughput, broker lag or stream backlog, cross-site tail latency, failure recovery, concurrent-job scaling, and policy-change responsiveness. RabbitMQ Streams deserves special attention in that setting because its local overhead may be outweighed by retained-stream and replay semantics once agents are no longer co-located.

VII. CONCLUSION

RQ1 asks which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS. At the level of isolated transport performance, client-streaming gRPC is the strongest result in the generic pipeline benchmark: it provides the highest sustained throughput on the clean matrix and the most stable pacing in the continuous scenario. That result establishes the value of direct streaming RPC as a technical baseline, while the brokered alternatives expose the cost and benefit of decoupling, persistence, and replay. The extended rerun adds an important qualification: a batched unary mapping recovers most of the streaming advantage and leads at the 256 KiB tier that matches real DYNAMOS chunks, so the decisive property for this workload is amortising per-message overhead, whether by streaming or by batching, rather than streaming itself.

RQ2 asks how streaming communication can be integrated into DYNAMOS while preserving dynamic service composition, policy-driven orchestration, and isolation. The implementation keeps the existing control path intact: requests are still evaluated and composed before execution, and generated jobs remain the execution unit. The change is in the bulk data path. Provider-side chunks must remain chunks through aggregate, algorithm, agent, and gateway stages; otherwise the system merely hides a full-buffered result behind a streaming interface.

RQ3 validates the implemented DYNAMOS variants on local bulk `dataThroughTtp` retrieval. All chunk-aware variants beat the classic unary baseline in median completion time and peak memory for 50k, 100k, and 250k row result sets. They also expose earlier partial results, which classic unary cannot do because its first result is its final result. Focused checks on `computeToData` and multi-provider `dataThroughTtp` show that the implementation is not confined to one request shape, although those checks remain concept-stage evidence. The strongest local recommendation is chunked unary. It gives the best 100k and 250k medians, the lowest 250k memory peak, and the smallest architectural shift from the original message-driven design.

The more general conclusion is boundary-specific. Intra-job gRPC streaming is useful where a direct stream between generated services is acceptable. RabbitMQ Streams is not the source of the local speedup by itself, because that speedup mainly comes from chunk preservation. Its stronger case is distributed inter-agent transfer, where retained records, replay, and independent consumers may matter more than local latency. The Scattered Directive and FABRIC context is therefore future work: it moves DYNAMOS toward cross-node VFL workflows, where network separation, broker placement, policy changes, and multi-party scaling can dominate local transfer costs.

The main research question asks how streaming communication can be integrated into DYNAMOS without weakening its policy-driven and adaptive character. The answer is to treat streaming as a data-path change after policy approval, not as a replacement for policy enforcement or orchestration. Compliance is preserved by construction in the evaluated implementation because the policy decision point, generated job composition, provider selection, and data-steward isolation remain unchanged. What changes is how approved bulk results are transferred once a job is already allowed to run. A stronger compliance claim, such as reacting to policy revocation during an active stream, remains future work.

APPENDIX A BENCHMARK PRESET DEFINITIONS

This appendix documents the complete preset definitions used in the benchmark evidence reported in the thesis. Each preset specifies the workload family, payload configuration, concurrency level, target rate where applicable, failure injection parameters, and resource overlay tier. The four reported presets are: *synthetic-clean* for bulk transfer, *synthetic-continuous* for steady-rate pacing, *synthetic-backpressure* for slow-consumer overload, and *synthetic-recovery* for forced-restart resilience. The harness also contains a CSV replay mode, but those results are not used in the current thesis argument.

APPENDIX B FULL RUN MATRIX

This appendix summarizes the canonical run scopes used in the current thesis. The body of the thesis presents the most

Table XIV
BENCHMARK PRESETS USED IN THE CURRENT TRANSPORT STUDY.

Preset	Key parameters	Purpose
<i>synthetic-clean</i>	Synthetic payloads from 256 B to 16 KiB in the completed corpus, extended to 256 KiB in the rerun harness; concurrency 1–16, no injected failure	Baseline bulk-transfer throughput, latency, and correctness matrix
<i>synthetic-continuous</i>	1 KiB payloads, 1,000 msg/s target, concurrency 4 and 8, one completed comparable run	Steady-rate pacing and jitter comparison
<i>synthetic-backpressure</i>	1 KiB payloads, 4,000 msg/s target, concurrency 8, 2 ms sink delay, one completed comparable run	Slow-consumer overload and queueing behaviour
<i>synthetic-recovery</i>	4 KiB payloads, 2,000 msg/s target, concurrency 8, transformer restart after 3 s, 120 retries at 250 ms backoff, one completed comparable run	Restart recovery, replay, and continuity analysis

decision-relevant subsets, while the benchmark result directory stores the corresponding per-run JSON, NDJSON, and CSV exports generated by the execution scripts. The chapter is based on the single-node five-transport *synthetic-clean* matrix, the single-node diagnostic scenario suite, and the three-broker clustered scenario runs with three repetitions per transport per scenario. The final rerun harness adds batched-unary gRPC and a 256 KiB payload tier, but those rerun results are not yet part of the reported corpus.

Table XV
EXECUTION SCOPE OF THE CURRENT BENCHMARK CORPUS.

Scope	Transports	Variable dimensions	Reps	Role in thesis
Single-node clean matrix	Client-streaming, unary, RabbitMQ, NATS, Kafka	Payload 256 B–16 KiB, concurrency 1–16	1	Full bulk-transfer baseline (Section III)
Single-node continuous	All five	Concurrency 4 and 8, 1 KiB, 1,000 msg/s	1	Pacing and jitter evidence
Single-node backpressure	All five	1 KiB, concurrency 8, 1 4,000 msg/s, 2 ms sink delay	1	Overload and queueing evidence
Single-node recovery	All five	4 KiB, concurrency 8, 1 transformer restart	1	Recovery and correctness evidence
Clustered clean (3-broker)	RabbitMQ, NATS, Kafka	4 KiB, concurrency 8	3	Clustered cross-run bulk-transfer consistency (Section III)
Clustered continuous (3-broker)	RabbitMQ, NATS, Kafka	1 KiB, concurrency 8, 1,000 msg/s	3	Clustered pacing under broker replication
Clustered backpressure (3-broker)	RabbitMQ, NATS, Kafka	1 KiB, concurrency 8, 1 4,000 msg/s, 2 ms delay	3	Clustered overload behaviour
Clustered recovery (3-broker)	RabbitMQ, NATS, Kafka	4 KiB, concurrency 8, 1 transformer restart	3	Clustered recovery correctness and variability

APPENDIX C DETAILED CLEAN-MATRIX RESULTS

This appendix records the full completed five-transport *synthetic-clean* matrix at the per-case level. The main chapter uses averages by transport and message size to keep the argument readable, but the tables below expose the exact throughput and latency values for every payload-size and concurrency combination in the current single-repeat suite.

APPENDIX D RESOURCE UTILISATION DETAILS

This appendix summarizes resource metrics for the diagnostic Kubernetes scenario suite where CPU and memory data are most informative. For brokered modes, the broker is reported as a separate role because its buffering layer is

Table XVI
DETAILED SYNTHETIC-CLEAN MESSAGE THROUGHPUT (MSG/S) BY
PAYLOAD SIZE AND CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	76,468	637	3,058	19,548	15,768
	4	118,209	1,899	25,812	24,292	10,876
	8	66,768	3,530	21,838	5,757	12,565
	16	40,970	4,279	1,553	28,496	10,660
1 KiB	1	28,171	696	17,810	16,976	10,760
	4	36,448	2,161	23,577	15,253	9,688
	8	28,743	3,163	1,045	6,350	5,103
	16	17,368	4,375	15,091	3,601	4,740
4 KiB	1	16,424	673	5,975	11,101	6,377
	4	19,838	1,834	8,928	11,211	6,903
	8	15,930	2,945	8,859	12,002	7,865
	16	9,239	3,823	7,860	5,360	4,965
16 KiB	1	2,872	635	2,233	4,497	2,737
	4	9,487	1,570	769	3,866	2,866
	8	9,138	1,698	466	6,507	2,797
	16	4,083	2,745	899	5,257	3,000

Table XVII
DETAILED SYNTHETIC-CLEAN DATA THROUGHPUT (MiB/s) BY PAYLOAD
SIZE AND CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	18.67	0.16	0.75	4.77	3.85
	4	28.86	0.46	6.30	5.93	2.66
	8	16.30	0.86	5.33	1.41	3.07
	16	10.00	1.04	0.38	6.96	2.60
1 KiB	1	27.51	0.68	17.39	16.58	10.51
	4	35.59	2.11	23.02	14.90	9.46
	8	28.07	3.09	1.02	6.20	4.98
	16	16.96	4.27	14.74	3.52	4.63
4 KiB	1	64.16	2.63	23.34	43.36	24.91
	4	77.49	7.16	34.88	43.79	26.97
	8	62.23	11.50	34.61	46.88	30.72
	16	36.09	14.93	30.70	20.94	19.39
16 KiB	1	44.88	9.92	34.89	70.27	42.76
	4	148.24	24.54	12.01	60.41	44.78
	8	142.78	26.53	7.28	101.67	43.71
	16	63.79	42.89	14.05	82.14	46.88

a core part of the transport cost. The backpressure preset produced too few stable `kubectl top` samples to support a meaningful per-role comparison across all transports, so the appendix focuses on the currently available continuous and recovery scenarios across all five transports. A clean-matrix broker CPU summary is intentionally omitted from the primary ranking because the clean-matrix argument is based on throughput, latency, and correctness, while resource pressure is discussed from the diagnostic scenarios with consistent role-level sampling.

Table XVIII
DETAILED SYNTHETIC-CLEAN p_{95} LATENCY (MS) BY PAYLOAD SIZE AND
CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	44.4	1.9	1,699.4	602.1	205.8
	4	63.7	3.2	481.8	457.1	1,112.3
	8	97.6	2.2	590.8	3,296.7	999.7
	16	227.1	3.9	1,111.3	422.6	941.1
1 KiB	1	31.6	1.8	314.2	661.1	476.4
	4	183.9	1.6	496.3	955.7	797.5
	8	185.0	2.9	1,833.1	2,773.8	3,511.2
	16	681.1	4.5	786.6	5,249.4	3,607.1
4 KiB	1	23.3	1.7	1,526.0	979.1	768.8
	4	66.0	3.1	998.3	1,160.2	1,438.7
	8	333.7	2.4	1,173.3	1,023.2	1,304.8
	16	1,067.7	4.1	1,589.8	3,256.2	3,039.3
16 KiB	1	35.0	1.5	1,001.1	2,226.7	138.5
	4	153.8	3.3	13,731.6	3,890.3	3,515.6
	8	300.5	4.7	407.2	1,933.2	2,789.9
	16	390.7	5.5	6,236.1	2,448.1	3,022.5

Table XIX
DETAILED SYNTHETIC-CLEAN p_{99} LATENCY (MS) BY PAYLOAD SIZE AND
CONCURRENCY.

Msg size	Concur.	Stream	Unary	RabbitMQ	NATS	Kafka
256 B	1	77.7	12.2	1,700.3	616.4	311.0
	4	65.8	15.5	507.5	501.6	1,210.1
	8	101.0	27.3	622.2	3,320.0	1,115.3
	16	266.6	48.5	1,123.1	487.0	1,097.6
1 KiB	1	57.0	13.1	338.3	723.4	484.5
	4	190.2	4.8	534.5	965.1	814.8
	8	249.8	41.2	1,909.0	2,865.2	3,624.5
	16	745.8	36.1	812.4	5,302.7	4,009.9
4 KiB	1	39.0	14.7	1,573.9	1,015.6	791.9
	4	76.3	23.9	1,191.1	1,182.2	1,516.1
	8	399.6	53.7	1,220.7	1,103.0	1,591.3
	16	1,379.0	64.8	1,934.5	3,348.0	3,167.0
16 KiB	1	44.8	15.4	1,041.4	2,361.8	294.2
	4	188.2	28.7	14,750.1	3,952.5	3,800.5
	8	347.6	33.9	598.8	2,012.5	3,173.7
	16	2,837.1	65.9	6,991.1	2,782.5	3,420.5

REFERENCES

- [1] J. Stutterheim, "Dynamos: Dynamically adaptive microservice-based os: A middleware for data exchange systems," Master's thesis, University of Amsterdam, 2023.
- [2] J. Stutterheim, A. Mifsud, and A. Oprea, "Dynamos: Dynamic microservice composition for data-exchange systems, lessons learned," in *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*, 2023.
- [3] C. Starck and J. Ghofrani, "Improving load balancing of long-lived streaming rpcs for grpc-enabled inter-service communication," in *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*, ser. CEUR Workshop Proceedings, S. Böhm and D. Lübke, Eds., vol. 3386, Hannover, Germany, 2023, pp. 45–51. [Online]. Available: <https://ceur-ws.org/Vol-3386/>
- [4] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar,

Table XX
CONTINUOUS SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Concur.	Producer	Transformer	Sink	Broker
Client-streaming	4	23.40 / 23.40	8.99 / 24.20	5.61 / 14.90	–
Client-streaming	8	–	9.31 / 25.60	5.64 / 15.50	–
Unary	4	35.70 / 35.70	17.88 / 39.10	11.12 / 24.10	–
Unary	8	–	15.54 / 33.70	9.72 / 20.90	–
RabbitMQ Streams	4	7.92 / 8.30	9.87 / 15.60	6.91 / 10.90	29.88 / 43.30
RabbitMQ Streams	8	7.88 / 8.20	10.46 / 15.80	7.50 / 11.70	33.00 / 46.50
NATS JetStream	4	–	12.62 / 24.90	5.60 / 15.40	18.37 / 40.90
NATS JetStream	8	–	16.23 / 23.70	6.45 / 14.20	22.74 / 38.50
Kafka	4	–	4.68 / 5.70	0.69 / 1.90	51.52 / 52.90
Kafka	8	–	8.58 / 8.80	1.00 / 2.20	41.13 / 58.80

Table XXI
RECOVERY SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	31.60 / 31.60	12.64 / 31.60	3.89 / 17.50	–
Unary	4.60 / 6.00	2.45 / 9.10	2.56 / 6.10	–
RabbitMQ Streams	8.84 / 10.20	13.38 / 17.10	7.51 / 13.90	31.76 / 49.90
NATS JetStream	0.30 / 0.30	17.29 / 25.20	7.28 / 16.20	20.99 / 43.50
Kafka	0.40 / 0.40	7.63 / 8.10	1.44 / 2.50	46.77 / 75.40

and S. Kalambur, “Benchmarking grpc protocol on various virtualization technologies,” in *ICT4SD 2025*, ser. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, vol. 1654, pp. 316–329.

- [5] A. G. Ibrahim, R. P. Lopes, J. Rufino, and P. Leitão, “On the impact of message brokers implementations in the choreography of microservices,” in *Open Lecture Series 2A (OL2A 2025)*, ser. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026.
- [6] J. Nuikka, “Comparison of cloud native messaging technologies,” Master’s thesis, Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. [Online]. Available: <https://urn.fi/URN:NBN:fi:tuni-202104223314>
- [7] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone, and S. Chakradhar, “Dataxc: Flexible and efficient communication in microservices-based stream analytics pipelines,” in *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCom / CyberSciTech*, 2022.
- [8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, “Benchmarking distributed stream data processing systems,” in *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1519–1530.
- [9] S. Henning and W. Hasselbring, “How to measure scalability of distributed stream processing engines?” in *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE ’21)*. ACM, 2021, pp. 85–88.
- [10] S. Ris, J. Araujo, and D. Beserra, “A systemic mapping of methods and tools for performance analysis of data streaming with containerized microservices architecture,” in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, 2023, pp. 1–6.
- [11] B. Çiftçi and B. Çiloğlu, “A review of comparative studies on performance evaluation of communication mechanisms for microservices,” in *Computational Science and Its Applications – ICCSA 2025*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland AG, 2025, vol. 15650, pp. 98–113.
- [12] J. Figueira and C. Coutinho, “Developing self-adaptive microservices,” *Procedia Computer Science*, vol. 232, pp. 264–273, 2024.
- [13] S. Neumaier, G. Havur, and T. Pellegrini, “Towards an architecture for policy-aware decentral dataset exchange,” in *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. [Online]. Available: https://www.researchgate.net/publication/358816711-Towards_an_Architecture_for_Policy-Aware_Decentral_Dataset_Exchange
- [14] G. Thiagarajan, V. Bist, and P. Nayak, “Strengthening grpc security in microservices: A proxy-based approach for mtls, jwt, and rbac

Table XXII
RECOVERY SCENARIO MEMORY USAGE BY ROLE (AVG./PEAK MiB).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	7.00 / 7.00	3.40 / 7.00	2.33 / 7.00	–
Unary	7.00 / 7.00	3.08 / 7.00	4.53 / 7.00	–
RabbitMQ Streams	10.00 / 10.00	25.72 / 31.00	5.78 / 8.00	490.91 / 511.00
NATS JetStream	8.00 / 8.00	14.00 / 14.00	9.75 / 15.00	306.06 / 537.00
Kafka	10.00 / 10.00	8.00 / 8.00	6.53 / 9.00	449.71 / 478.00

enforcement,” *International Journal of Computer Applications*, vol. 187, no. 28, Aug. 2025.

- [15] J. A. Rasheedh and S. Saradha, “Reactive microservices architecture using a framework of fault tolerance mechanisms,” in *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021.
- [16] B. Guntupalli and S. Vamshi Ch, “Designing microservices that handle high-volume data loads,” *International Journal of AI, Big Data, Computational and Management Studies*, vol. 4, no. 4, pp. 76–87, 2023.
- [17] Y. Gan and C. Delimitrou, “The architectural implications of cloud microservices,” *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.
- [18] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, “Logplayer: Fault-tolerant exactly-once delivery using grpc asynchronous streaming,” 2019, arXiv preprint.
- [19] Ritu, S. Arora, A. Bhardwaj, A. Kukkar, and S. Kaur, “A comparative analysis of communication efficiency: Rest vs. grpc in microservice-based ecosystems,” in *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024.
- [20] V. Lähdeväoja, “Communication methods and protocols between microservices on a public cloud platform,” Master’s thesis, Aalto University, School of Science, Espoo, Finland, May 2021. [Online]. Available: <https://urn.fi/URN:NBN:fi:aalto-202106207501>
- [21] gRPC Authors. (2024) Core concepts, architecture and lifecycle. Last modified 2024-11-12. [Online]. Available: <https://grpc.io/docs/what-is-grpc/core-concepts/>
- [22] Apache Software Foundation. (2026) Apache kafka. [Online]. Available: <https://kafka.apache.org/>
- [23] P. Dobbelaere and K. Sheykh Esmaili, “Kafka versus rabbitmq,” 2017, arXiv preprint.
- [24] M. Mohammad, “Analysis of design patterns and benchmark practices in apache kafka event-streaming systems,” 2025, arXiv preprint.
- [25] T. Sharvari and K. Sowmya Nag, “A study on modern messaging systems: Kafka, rabbitmq and nats streaming,” 2019, arXiv preprint.
- [26] RabbitMQ Team. (2026) Streams and superstreams (partitioned streams). [Online]. Available: <https://www.rabbitmq.com/docs/streams>
- [27] The NATS Authors. (2026) Overview. [Online]. Available: <https://docs.nats.io/nats-concepts/overview>
- [28] —. (2026) Request-reply. [Online]. Available: <https://docs.nats.io/nats-concepts/core-nats/reqreply>
- [29] —. (2026) Jetstream. [Online]. Available: <https://docs.nats.io/nats-concepts/jetstream>
- [30] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl, and R. Rabiser, “A comprehensive benchmarking analysis of fault recovery in stream processing frameworks,” in *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems (DEBS ’24)*. ACM, 2024, pp. 171–182.
- [31] DYNAMOS Contributors, “DYNAMOS source repository, main branch, commit a2d9531,” <https://github.com/neburpoots/DYNAMOS/tree/a2d953166c369260e6014f1bd988ee3ae45592d>, 2026, accessed: 2026-06-09.
- [32] —, “DYNAMOS streaming implementations prototype, feature/streaming-implementations branch, commit 923b018,” <https://github.com/neburpoots/DYNAMOS/tree/923b018e622e30ed6141a3e9c925417ce9150e5d>, 2026, accessed: 2026-05-25.
- [33] J. Jongejans, A. Koufakis, and A. Opreescu, “Vertical federated learning on scattered directive: Enforcing policies on VFL workflows,” in *Proceedings of the 2025 IEEE International Conference on Big Data (BigData)*, 2025, pp. 267–274.

- [34] DYNAMOS-UVA, “Scattered directive source repository,” <https://github.com/DYNAMOS-UVA/Scattered-Directive>, 2026, accessed: 2026-05-25.
- [35] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, K.-C. Wang, T. Lehman, P. Ruth, and E. Deelman, “FABRIC: A national-scale programmable experimental network infrastructure,” *IEEE Internet Computing*, vol. 23, no. 6, pp. 38–47, 2019.
- [36] FABRIC Testbed, “About FABRIC,” <https://portal.fabric-testbed.net/about/about-fabric>, 2026, accessed: 2026-05-25.